

Part 01 — Essay Questions

Question 1: Why can't a struct inherit from another struct or class in C#?

Answer:

A struct in C# is a value type, while classes are reference types. Structs cannot inherit from another struct or class because C# does not support implementation inheritance for structs. A struct can only implicitly inherit from `System.ValueType`, which itself inherits from `System.Object`.

However, a struct can implement one or more interfaces. This allows structs to provide specific behavior without using class inheritance.

Question 2: How do access modifiers impact the scope and visibility of a class member?

Answer:

Access modifiers determine where a class member can be accessed in a C# program. They control the visibility and accessibility of fields, methods, properties, and other members.

The main access modifiers include:

- **private:** The member can only be accessed inside the same class or struct.
- **internal:** The member can be accessed from anywhere within the same assembly or project.
- **public:** The member can be accessed from anywhere.
- **protected:** The member can be accessed inside the class and by derived classes.
- **protected internal:** The member can be accessed from the same assembly or from derived classes.
- **private protected:** The member can be accessed by derived classes within the same assembly.

Access modifiers help control how different parts of a program interact with each other and protect internal implementation details.

Question 3: Why is encapsulation critical in software design?

Answer:

Encapsulation is critical because it protects an object's internal data by restricting direct access to its fields and providing controlled ways to access or modify that data.

For example, instead of allowing a field such as `Salary` to be directly modified, it can be declared as `private` and accessed through methods or properties.

Encapsulation provides several benefits:

- Protects data from unwanted modification.
- Allows validation before changing data.
- Hides implementation details.
- Makes code easier to maintain.
- Reduces dependencies between different parts of the program.
- Improves security and reliability.

Therefore, encapsulation is an important principle of Object-Oriented Programming.

Question 4: What are constructors in structs?**Answer:**

A constructor in a struct is a special method used to initialize the fields of the struct when an object is created.

Structs can have parameterized constructors that allow developers to initialize their fields with specific values.

Question 5: How does overriding methods like `ToString()` improve code readability?**Answer:**

Overriding `ToString()` allows an object to provide a meaningful string representation of its data.

By default, `ToString()` may not provide useful information about the object's contents. By overriding it, we can define exactly how the object should be displayed.

Question 6: How does memory allocation differ for structs and classes in C#?

Answer:

Structs are value types, while classes are reference types.

A struct variable normally contains its data directly. When a struct is assigned to another variable or passed by value to a method, its value is copied.

A class object is created as an object in managed memory, and a variable of the class type contains a reference to that object. When a class variable is assigned to another variable, the reference is copied, so both variables can refer to the same object.

Therefore, modifying a copied struct does not normally affect the original struct, while modifying an object through another reference can affect the same class object.

Part 02 — Essay Questions

Question 7: What is a Copy Constructor?

Answer:

A copy constructor is a constructor that creates a new object by copying the values from an existing object of the same class.

It receives an object of the same class as a parameter and copies its fields or properties into the new object.

Question 8: What are constructors and their types?

Answer:

A constructor is a special member of a class or struct that is automatically called when an object is created. Its main purpose is to initialize the object's data.

Common types of constructors in C# include:

1. **Default / Parameterless Constructor**
A constructor that does not take parameters and initializes an object with default values.
2. **Parameterized Constructor**
A constructor that accepts parameters and uses them to initialize the object's fields or properties.
3. **Copy Constructor**
A constructor that creates a new object by copying the data from an existing object of the same type.

4. **Static Constructor**

A special constructor used to initialize static members of a class. It runs automatically and does not take parameters.

5. **Private Constructor**

A constructor declared as `private`. It prevents other classes from directly creating objects of that class and can be used in designs such as Singleton or classes that provide only static functionality.

Question 9: What is an Indexer, and when is it used? Mention business cases where you have to utilize it.

Answer:

An indexer is a C# feature that allows an object to be accessed using an index, similar to accessing elements of an array.

Indexers are useful when a class represents a collection of objects and we want to access those objects using an index or key.

Business use cases include:

- A shopping cart where products can be accessed by their position or ID.
- An employee management system where employees can be accessed by an index.
- A student management system where students can be accessed by their index.
- A dictionary-like business object where data can be accessed using a key.
- A configuration system where settings can be accessed using a key.

Indexers make collection-like classes easier and more natural to use.

Question 10: Summarize the keywords we have learned last lecture.

Answer:

The main C# keywords and concepts learned in the lecture include:

- **struct:** Defines a value type that is useful for representing small data structures.
- **class:** Defines a reference type used to create objects.
- **public:** Makes a member accessible from anywhere.
- **private:** Restricts access to the containing class or struct.
- **internal:** Allows access within the same assembly or project.
- **protected:** Allows access inside the class and its derived classes.
- **this:** Refers to the current object and is also used when defining indexers.
- **new:** Creates a new object or initializes a value type.

- **ref:** Passes an argument by reference so the method can modify the original variable.
- **override:** Allows a derived implementation to replace a virtual or overridden member from a base type.
- **constructor:** Initializes an object when it is created.
- **indexer:** Allows an object to be accessed using an index like an array.